LESSON PLAN · SCRATCH (GR 6–8) INTERMEDIATE: DATA & SIMULATIONS

Lesson 5: Decomposition and Abstraction

Module 2 | 45 minutes | Decomposition and refactoring | Grades 6–8 · ages 11–14

Objective: Break a problem into smaller procedures, build a solution from them, and refactor repeated code into a procedure.

Success criteria

- I can decompose a problem into procedures.
- I can build bigger procedures from smaller ones.
- I can refactor repeated code.

Key vocabulary

- **decomposition**: Breaking a problem into smaller parts.
- **abstraction**: Hiding details behind a simple name.
- **refactor**: Improve code structure without changing behaviour.
- **bottom-up**: Build small parts first, then combine.

You will need

- Projector with Kitabash (lesson video, Code Lab)
- Devices: 1 per learner or pair
- Printed worksheet (optional)

Timed lesson plan (45 min)

Time	Stage	What happens
5 min	**Warm-up (unplugged)**	Decompose breakfast (5 min): break “make breakfast” into procedures with parameters.
5 min	**Watch**	Play the lesson video with captions; pause at each heading and ask a question.
8 min	**Build along**	A street of houses: learners follow the steps.
14 min	**Perform**	Code Lab: street of houses, Code Lab: refactor with a procedure
9 min	**Probe**	Decompose “make a racing game” — which part is which?, activity
4 min	**Wrap-up**	Exit ticket: What does refactoring change, and what must it NOT change?

Unplugged warm-up

Decompose breakfast (5 min): break “make breakfast” into procedures with parameters.

Teaching the idea

Decomposition splits problems into sub-problems; procedures implement them bottom-up; abstraction lets you use a procedure by name; refactoring improves structure without changing behaviour (verified by tests).

Analogy: Building with LEGO: bricks → walls → rooms → a house.

Live demo: Refactor the frog live: count blocks before and after; re-run to prove behaviour is unchanged.

Questions to ask

Ask	Listen for
What is the smallest piece of the house?	A square or a roof.
How do you know a refactor worked?	Same behaviour, cleaner code.
Why build bottom-up?	Test small pieces first.

Differentiation

Support
Provide house as a call sequence; learners add the loop.

Stretch
Add a door (size) procedure and windows.

Common misconceptions

Misconception	How to fix it
Refactoring adds features	It keeps behaviour the same.
Bigger procedures are better	Small, focused procedures are easier to test.

Assessment

Use the Code Lab results (auto-checked), the quick checks and the Probe answers. Written answers arrive in your review queue.

Exit ticket
What does refactoring change, and what must it NOT change?

Homework
Find copy-pasted code in one of your old projects and plan a refactor.

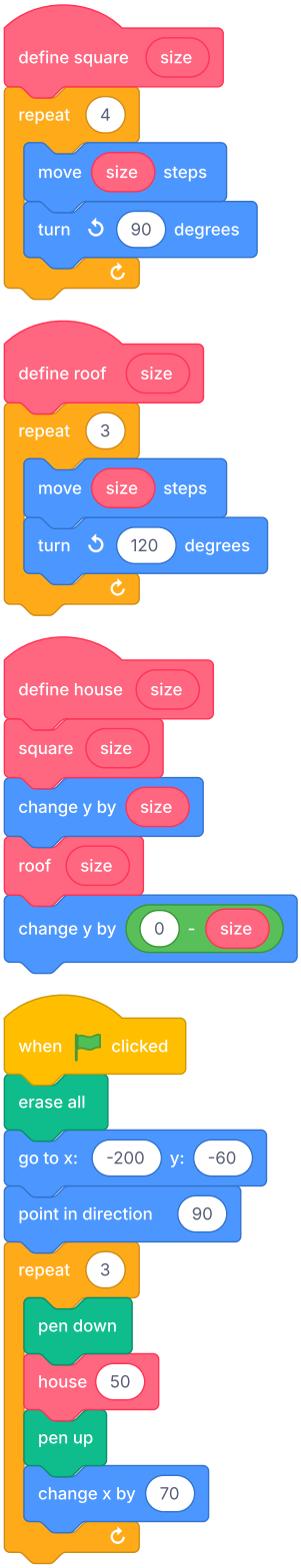
Answer key — Kitabash activities

1. Code Lab: street of houses (perform · code lab)

Goals checked: Uses the procedures · Three houses · Uses a loop

One solution:

Pen



2. Code Lab: refactor with a procedure (perform · code lab)

Goals checked: Same result · Uses a procedure · Shorter code

One solution:

Frog



3. Decompose "make a racing game" — which part is which? (probe · categorize)

Player: steer with arrow keys, speed up with the up arrow
Track: draw the road edges, place the finish line
Rules: lap counter, crash when touching the grass

4. Explain your thinking (probe · free response)

Look for: procedure, decompose, does. **Example:** A space game: setupLevel puts the asteroids in place; movePlayer reads the keys; fire creates a laser clone; checkHits tests collisions and updates the score. Each one is small and I can test it on its own.

Answer key — worksheet

1. **Key words:** decomposition = Breaking a problem into smaller parts. · abstraction = Hiding details behind a simple name. · refactor = Improve code structure without changing behaviour. · bottom-up = Build small parts first, then combine.
2. **Plan your code:** Goals: Uses the procedures; Three houses; Uses a loop.
3. **Plan your code:** Goals: Same result; Uses a procedure; Shorter code.
4. **Decompose "make a racing game" — which part is which?:** Player: steer with arrow keys, speed up with the up arrow · Track: draw the road edges, place the finish line · Rules: lap counter, crash when touching the grass

5. **Explain your thinking:** A space game: setupLevel puts the asteroids in place; movePlayer reads the keys; fire creates a laser clone; checkHits tests collisions and updates the score. Each one is small and I can test it on its own.

Kitabash original curriculum. Scratch is a project of the Scratch Foundation in collaboration with the MIT Media Lab; this course is not affiliated with or endorsed by them.